

Python | 

(Task 1 of 6) In this tutorial, we will learn about **mutable values**, illustrated with **arrays**.

The following program illustrates how to create arrays

Python [Run 

```
print([ [ 1, 2 ], [ 3 ], [ ] ]) console.log([ [ 1, 2 ], [ 3 ], [ ] ])
```

This program produces `[[1, 2], [3], []]`. It creates four arrays:

- a two-**element** array that refers 1 and 2
- a one-element array that refers 3
- an empty array
- a three-element array that refers all three aforementioned arrays

Python | 

(Task 2 of 6) What is the result of running this program?

Python [Run 

Pseudo

```
n = 44
def f(x):
    return x + 1
print([ n, f(n) ])

let n = 44
fun f(x):
    return x + 1
end
print(vec[n, f(n)])
```

[44, 45]

2

You predicted the output correctly 🎉🎉🎉

3

This program binds `n` to 44 and `f` to a function that adds 1 to its parameter. After that, the program creates a array that refers the value of `n`, which is 44, and the value of `f(n)`, which is 45. The array is printed as `[44, 45]`.

Click [here](#) to run this program in the Stacker.

Python | 

(Task 3 of 6) The following program illustrates how to refer to array elements.

Python [Run 

Scala 3

```
v = [ 84, 75 ]
print(v[0])
print(v[1])
print(v[2])

val v = (84, 75)
println(v(0))
println(v(1))
println(v(2))
```

This program produces 84 75 error. It refers to the 0-th (i.e., first) element, the 1-th

element, and then tries to refer to the 2-th element.

(Task 4 of 6) What is the result of running this program?

Python | 

Python 

JavaScript

```
v = [ [ 58, 43 ], [ 43, 66 ] ]    let v = [ [ 58, 43 ], [ 43, 66 ] ];
vr = v[1]                        let vr = v[1];
print(vr[0])                     console.log(vr[0]);
```

43

6

You predicted the output correctly 🎉🎉🎉

7

Recall that the left-most element is the 0-th (rather than 1-th!) element. `v[1]` produces the value of `[ 43, 66 ]`. So, `vr[0]` produces 43.

Click [here](#) to run this program in the Stacker.

(Task 5 of 6) The following program illustrates how to mutate arrays.

Python | 

Python 

Scala 3

```
m = [ 62, 77 ]    val m = Buffer(62, 77)
m[0] = 83         m(0) = 83
print(m[0])       println(m(0))
```

This program produces 83. It **mutates** the array by **replacing** the 0-th element with 83 and then refers to the 0-th element.

(Task 6 of 6) What is the result of running this program?

Python | 

Python 

Scala 3

```
x = [ 92, 73 ]    val x = Buffer(92, 73)
x[0] = 67         x(0) = 67
print(x)          println(x)
```

[67, 73]

10

You predicted the output correctly 🎉🎉🎉

11

`x` is bound to a array. The initial content of the array is 92 and 73. The mutation replaces

the first element with 67. A array referring 67 and 73 is printed as [67, 73].

Click [here](#) to run this program in the Stackr.

You have finished this tutorial 🎉🎉🎉

Please print the finished tutorial to a PDF file so you can review the content in the future. **Your instructor (if any) might require you to submit the PDF.**

Start time: 1760133586911